

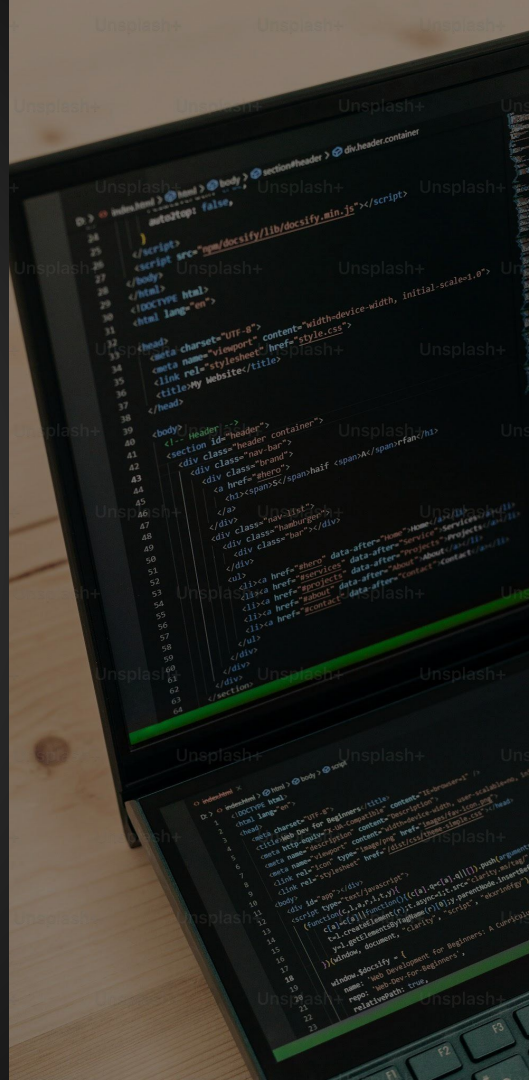
A photograph of a person's hand typing on a laptop keyboard. The laptop screen displays code. In the background, there is a desk with various items: a coffee cup, a calculator, a pen holder with pens, and a small container with a cup of coffee. The text "Desenvolvimento de Microserviços" is overlaid in large white letters.

Desenvolvimento de Microserviços

Prof. Guilherme Alves

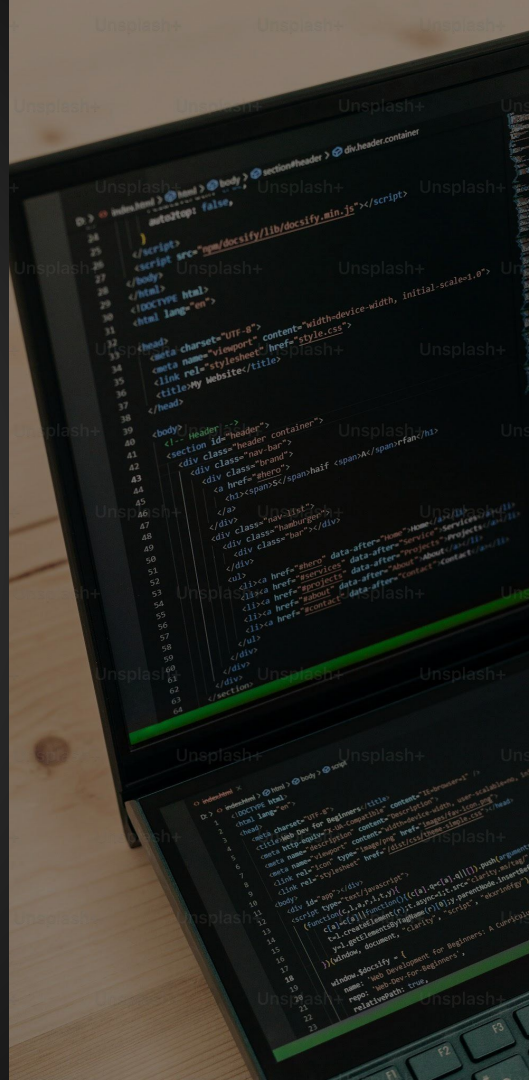
Boas Práticas e Desempenho em APIs

Quando falamos de boas práticas e desempenho em APIs, estamos nos referindo a um conjunto de princípios e técnicas que garantem que a **API** seja eficiente, segura, fácil de usar e manter.



Boas Práticas

- URLs intuitivas
- Uso correto de verbos HTTP
- Respostas padronizadas
- Tratamento adequado de erros
- Segurança
- Versionamento
- Documentação

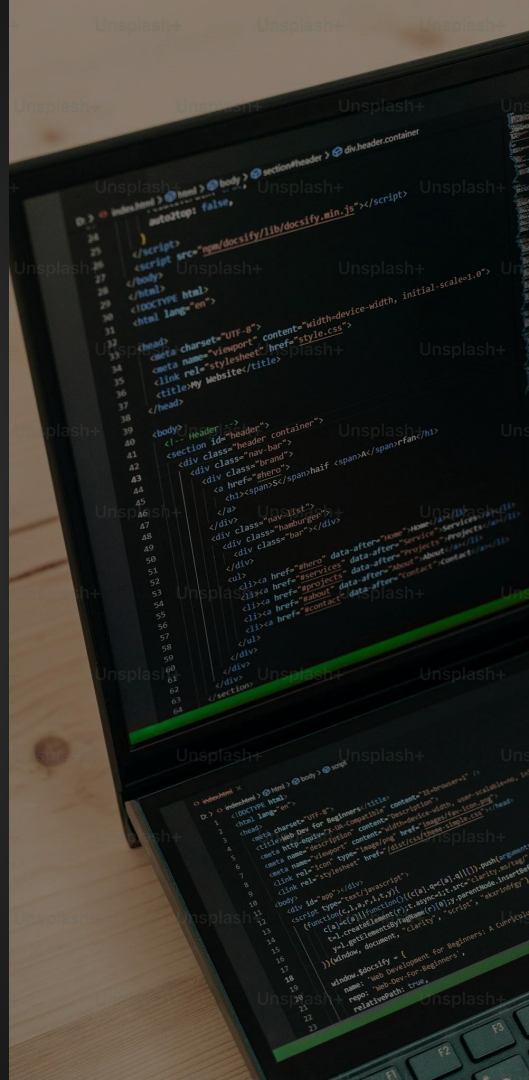


Nomenclatura

A estrutura da **URL** deve seguir um padrão consistente para facilitar a compreensão e manutenção da **API**.

<https://dominio/contexto/{{versão}}/{{coleção}}/{{id}}/{{subcoleção}}/>

- Evitar mais níveis que os demonstrados acima
- Sempre utilizar coleções no plural
- Utilizar *lowerCamelCase* para nomes compostos

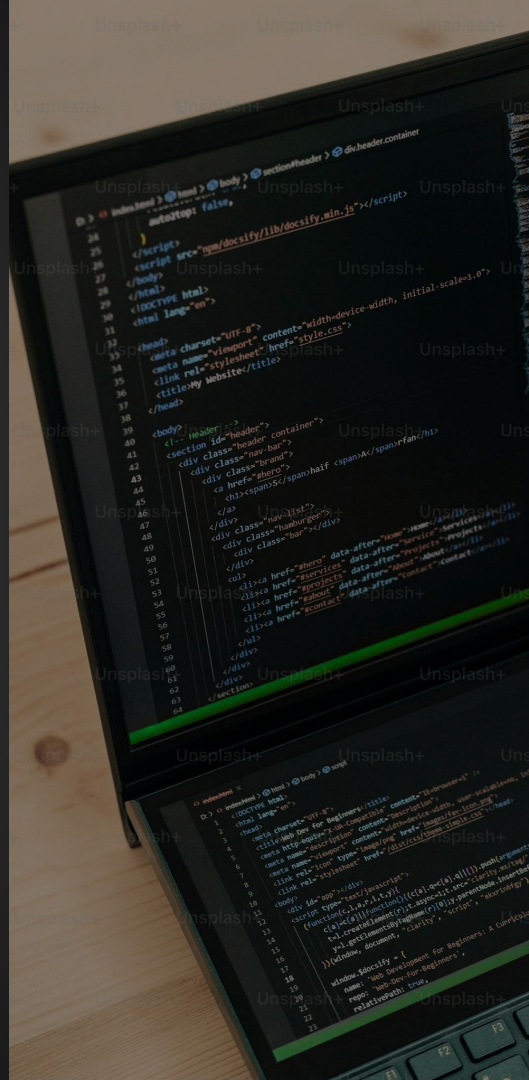


Verslonamento

O versionamento permite a evolução da API sem impactar integrações existentes.

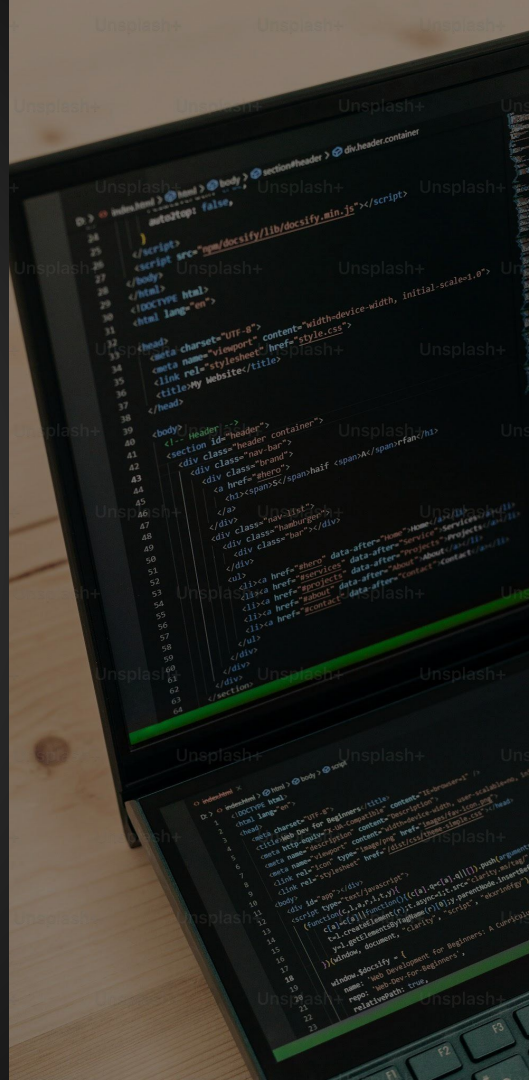
<https://dominio/v1/users>

- Evita quebra de compatibilidade
- Facilita a manutenção e a evolução da API
- Garante previsibilidade ao definir um ciclo de vida para cada versão



CRUD de Recursos

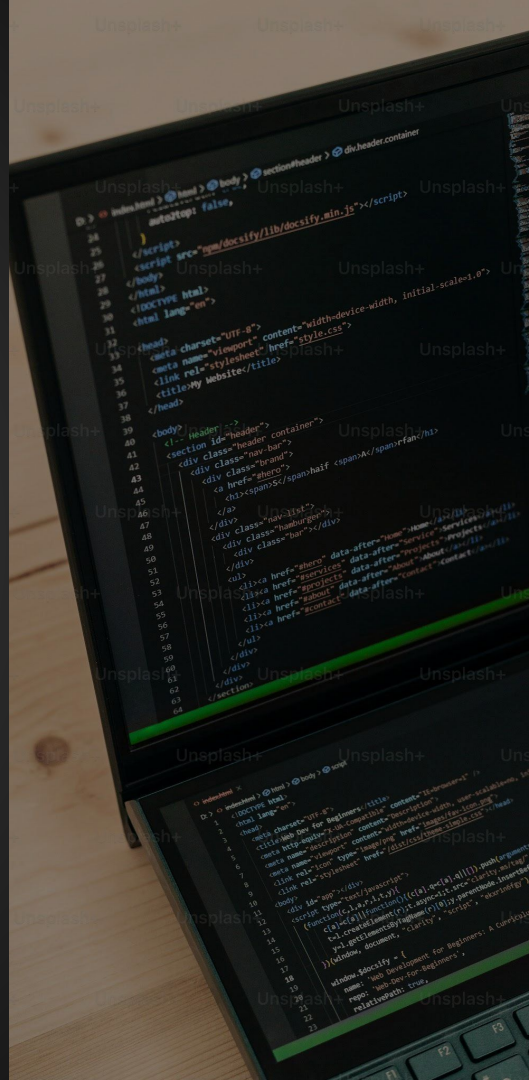
- **GET** `/users` - Retorna a lista de usuários
 - Não passar body parameters
- **GET** `/users/{id}` - Retorna um usuário específico
 - Não passar body parameters
- **POST** `/users` - Cria um novo usuário
 - Não passar query parameters
- **PUT** `/users/{id}` - Atualiza os dados de um usuário
 - Não passar query parameters
- **DELETE** `/users/{id}` - Remove um usuário
 - Não passar body parameters



Ações Específicas

Para operações que não são diretamente **CRUD**, recomenda-se o uso de sub-recursos ou verbos descritivos:

- **PATCH** `/users/{id}/activate` - Ativa um usuário
- **PATCH** `/orders/{id}/cancel` - Cancelar um pedido



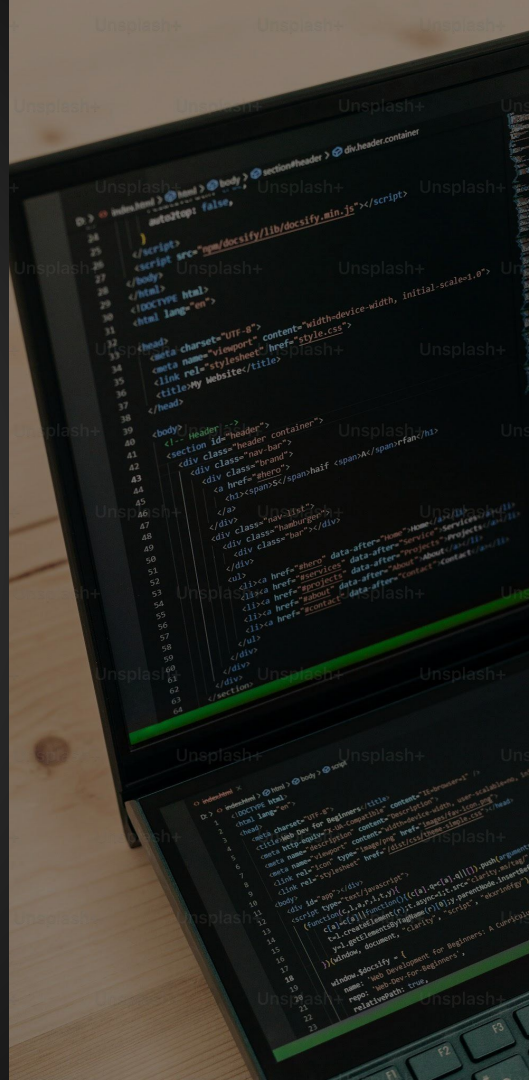
Parâmetros

Utilizar *snake case* em query params, se possível, para melhor legibilidade.

- **GET** `/users?sort_by=name`

Evitar verbos nos nomes dos recursos

- **POST** `/users/{id}/resetPassword` é melhor que `resetPasswordUser`

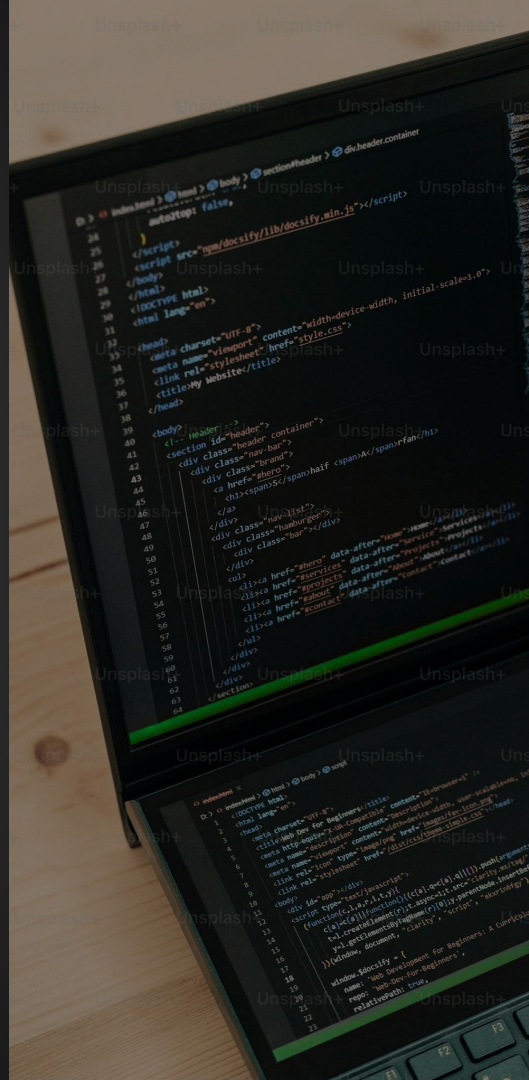


Campos Booleanos

Os campos booleanos devem ser usados de forma que o valor true indique algo **permitido**

- **allowAccess** ao invés de **accessDenied**
- **visitorsEnabled** ao invés de **visitorsDisabled**

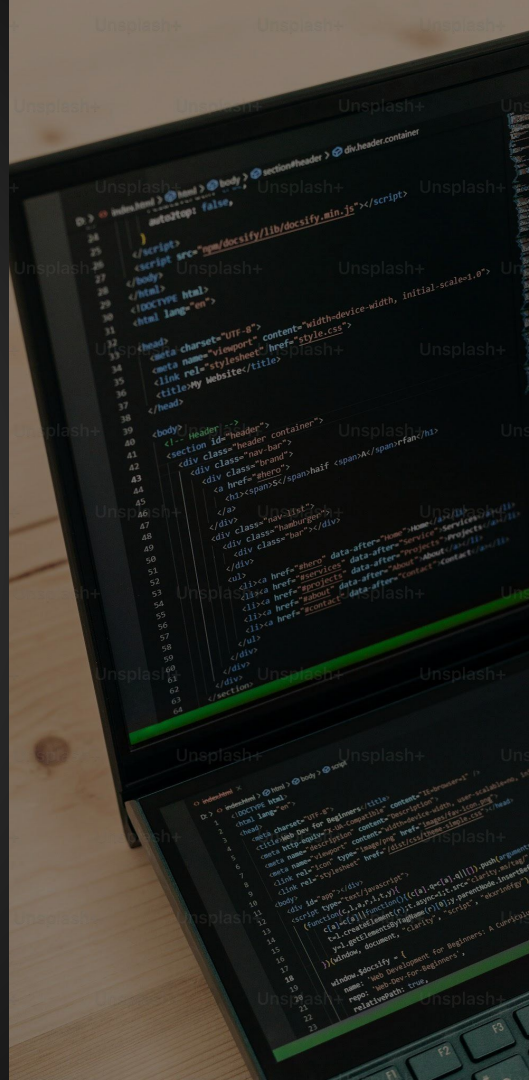
Valores nulos devem ser interpretados como *false* para evitar ambiguidades e garantir consistência no comportamento



Unidades de Medida

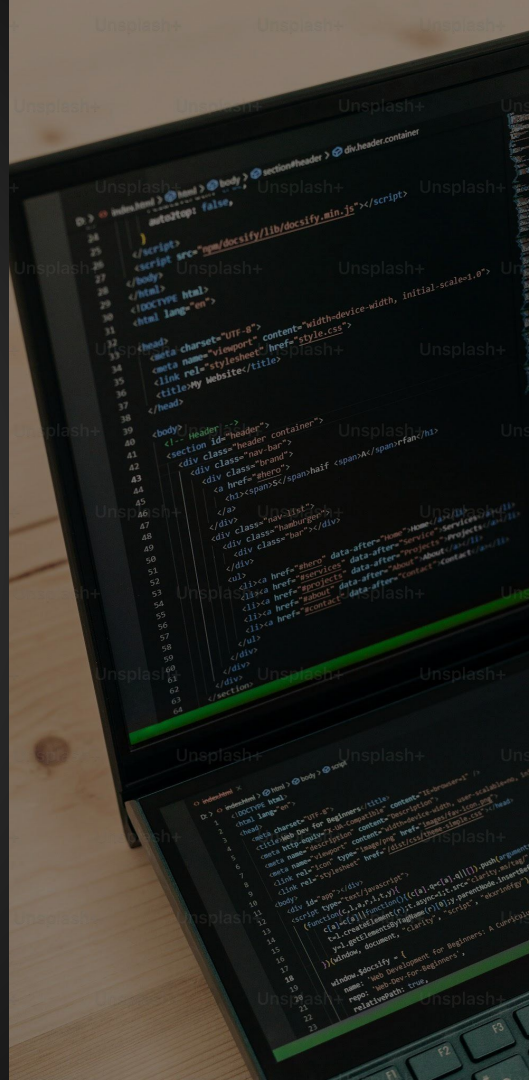
Para evitar ambiguidades, todas as unidades de medida devem ser explicitadas nos nomes dos atributos.

- **durationInMinutes** ao invés de **duration**
- **weightInKg** ao invés de **weight**
- **distanceInMeters** ao invés de **distance**



Status de Retorno

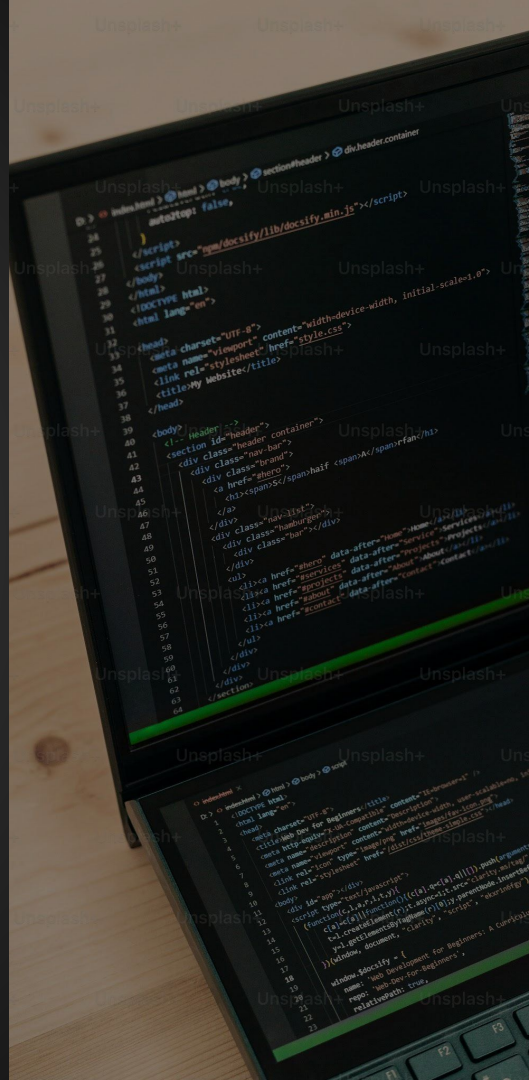
- **200 OK** - Requisição bem-sucedida
- **201 Created** - Recurso criado com sucesso
- **204 No Content** - Operação concluída sem retorno
- **400 Bad Request** - Erro na requisição do cliente
- **401 Unauthorized** - Falha de autenticação
- **404 Not Found** - Recurso não encontrado
- **500 Internal Server Error** - Erro interno no servidor



Tratamento de Erros

A padronização dos erros nas respostas da **API** traz diversos benefícios, incluindo

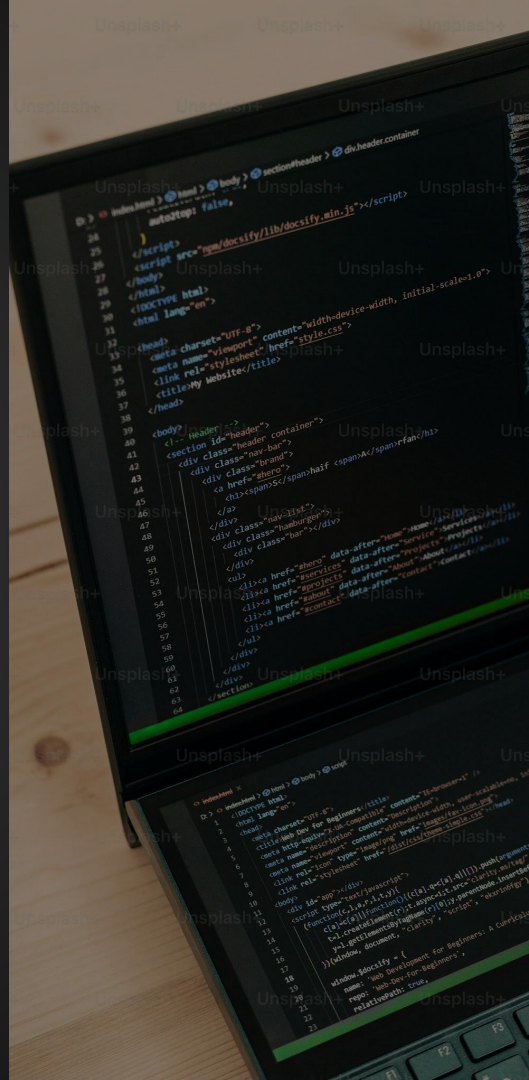
- Facilidade no tratamento de erros pelo cliente
- Consistência nas respostas
- Depuração mais eficiente
- Melhoria na experiência do usuário final



Listas

Para otimizar o uso de listas em APIs, recomenda-se seguir estas diretrizes

- Paginação
- Ordenação
- Filtros

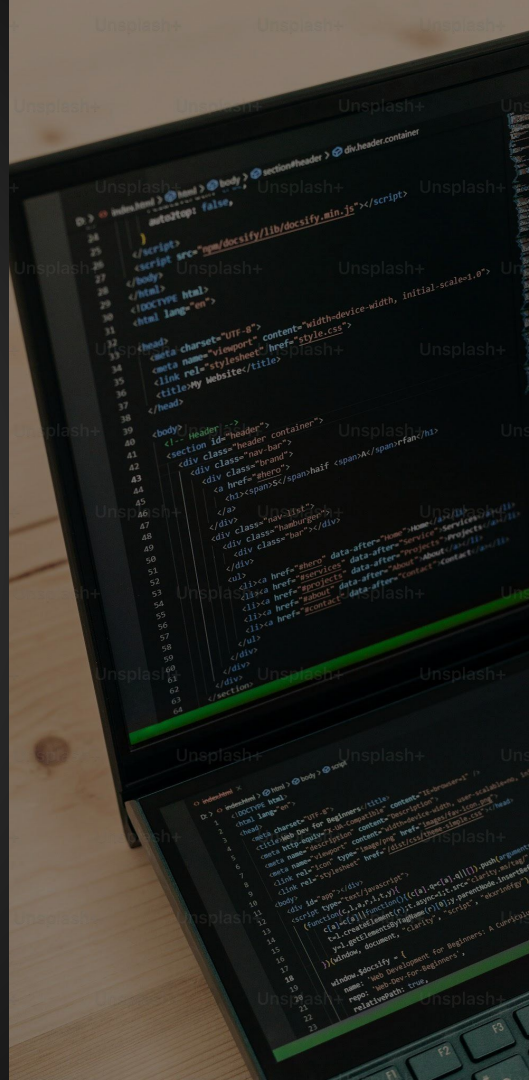


Listas - Paginação

A paginação evita que grandes volumes de dados sejam retornados de uma só vez, melhorando a performance da API. Os seguintes parâmetros devem ser usados

`GET /users?_page=2&_size=50`

- `_page` - Define a página desejada
- `_size` - Define o número de itens por página

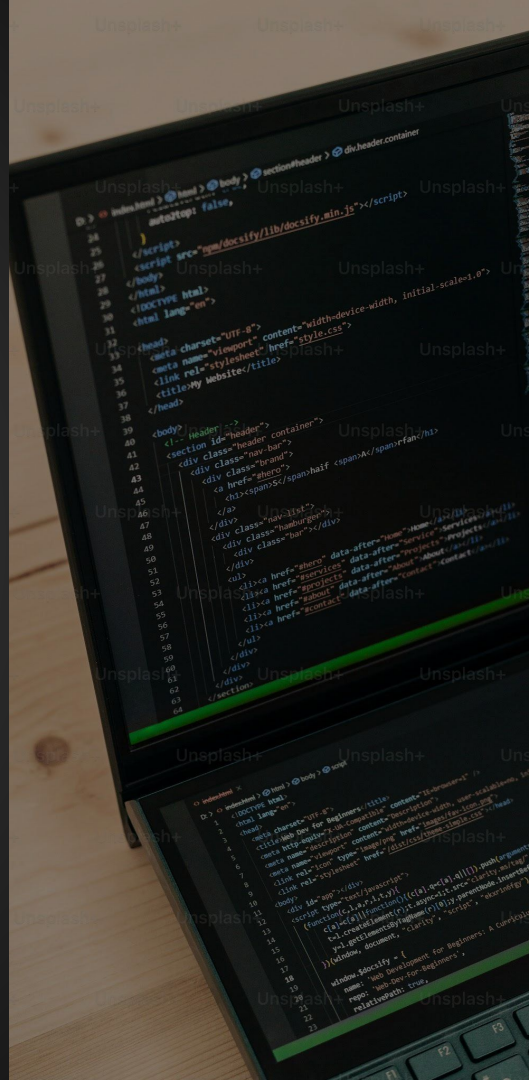


Listas - Ordenação

A ordenação permite que os resultados sejam retornados em ordem crescente ou decrescente

`GET /users?_order="city desc"`

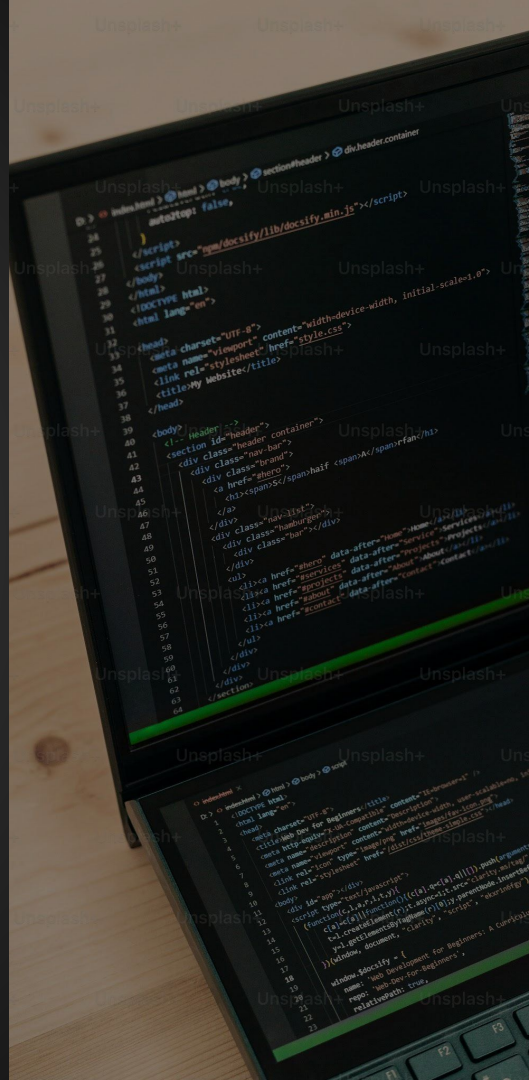
- `_order` - Define a ordem da lista
- Aceita valores `asc` (padrão) ou `desc`



Listas - Filtros

Os filtros devem ser passados via *query string* para facilitar a manipulação de dados

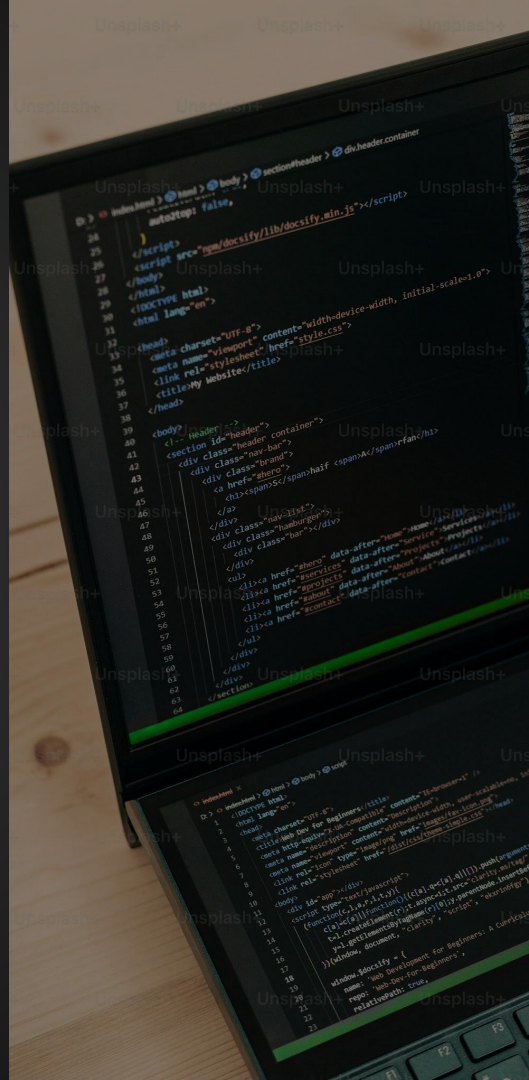
`GET /users?status=active&role=admin`



Segurança

Quando se trata de segurança em **APIs**, é importante implementar uma série de medidas para proteger os dados e garantir que apenas usuários autorizados possam acessar recursos sensíveis.

- Autenticação
- Autorização
- Criptografia
- Rate Limiting
- Validação de entrada
- CORS (Cross-Origin Resource Sharing)
- Auditoria
- Logs

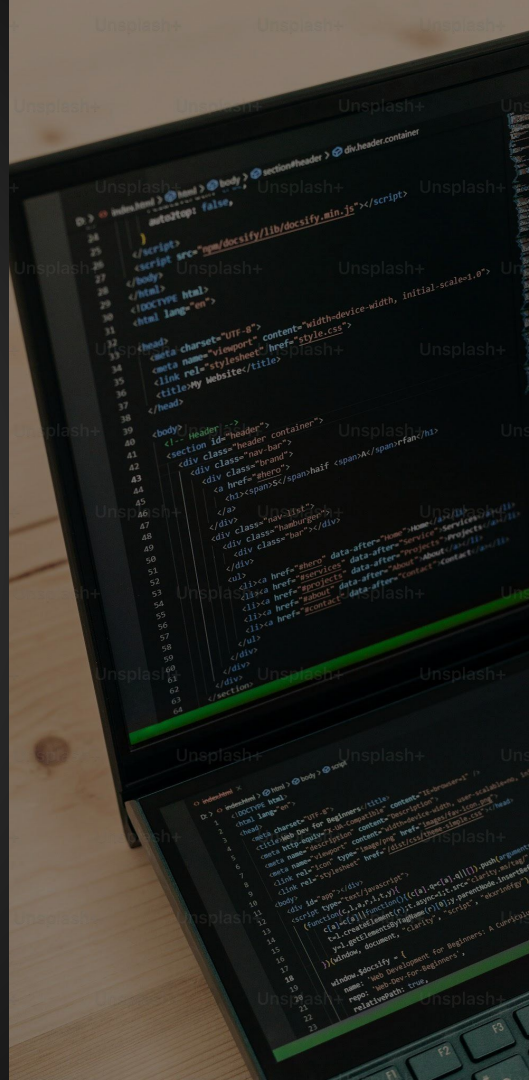


Conceito de Idempotência no desempenho

A idempotência garante que uma mesma requisição, mesmo se executada várias vezes, produza sempre o mesmo efeito no servidor.

- Métodos idempotentes - *GET, PUT, DELETE*
- Métodos **NÃO** idempotentes - *POST*

Métodos que são idempotentes são passíveis de serem cacheados.



Resumindo

São pequenos detalhes que em um contexto maior garantem que a **API** seja fácil de usar, segura e bem organizada.

